

InsightStream

AI-Native Product Requirements Document

Pranav Aadhithya Kalaibabu | Version 3.0 | Aakash Gupta's Framework | Feb 2026

Table of Contents

0. One-Liner	3
1. The Vision & The “Why”	3
1.1 The Pain Point (Quantified)	3
1.2 Why AI, Not Rules?	3
1.3 Why Now?	3
2.1 Primary Evaluation Matrix	4
2.2 Why This Matrix Matters	4
3.1 Pipeline Execution Model.....	6
4. Model Selection & Logic	7
4.1 Primary Model: Gemini 2.5 Flash	7
4.2 Approach: RAG, Not Fine-Tuning.....	7
4.3 RAG Implementation	8
5. AI UX & Trust.....	9
5.1 Explainability: The “Why” Behind Every Insight	9
5.2 Perceived Latency Strategy	9
5.3 The “I Don’t Know” UX	10
6. Failure Modes & Guardrails	11
6.1 Failure Taxonomy	11
6.2 Rejection Criteria: When the AI Says “No”	12
7. Success Metrics	13
7.1 North Star Business Metric.....	13
7.2 Technical Metric 1: Precision @ K (Grounding Attribution Rate).....	13
7.3 Technical Metric 2: Latency p95.....	13
7.4 Guardrail Metric: Manual Override Rate	13
8. Aakash Framework Compliance Checklist.....	14

0. One-Liner

InsightStream converts raw customer interview transcripts into validated, citation-backed product backlogs in under 45 seconds, eliminating the 4-hour manual synthesis tax while guaranteeing a 0% hallucination rate through adversarial semantic entailment.

1. The Vision & The “Why”

1.1 The Pain Point (Quantified)

A single 60-minute customer interview generates approximately 9,000 words of unstructured dialogue. The industry standard for synthesis: 4 hours per 1 hour of interview. For a PM running 10 interviews per quarter, that’s 160 hours per year spent on spreadsheet tagging, time that produces zero new product decisions.

The consequence is not just wasted time. It’s selective analysis. Teams analyze 2–3 “favorite” interviews and ignore the rest. Critical edge-case feedback from quieter participants is never synthesized. Product roadmaps are built on incomplete data.

1.2 Why AI, Not Rules?

A rule-based script would require:

1. Exhaustive keyword lists that break on novel vocabulary. A user saying “I wish I could just get a morning digest” would need “morning digest” pre-mapped to “daily summary email.” Every new interview introduces terms no keyword list anticipated.
2. Sentiment heuristics that fail on sarcasm. “The color scheme is nice? I’m kidding.” registers as positive sentiment in VADER/TextBlob. A probabilistic model understands the rhetorical inversion.
3. Static speaker assignment that assumes perfect diarization. Real transcripts have label drift, overlapping speech, and mid-sentence speaker swaps. A rule-based system has no mechanism to infer intent from context when labels are wrong.
4. Manual theme clustering that cannot scale. Grouping “the login is slow,” “auth speed is poor,” and “SSO takes forever” as the same theme requires semantic understanding, not string matching.

The core insight: extraction is easy, validation is hard. Any LLM can pull quotes from a transcript. The product differentiator is proving those quotes actually support the claimed insight — which requires a second, adversarial AI pass (semantic entailment) that no rule-based system can perform.

1.3 Why Now?

Gemini 2.5 Flash achieves structured JSON output with function-calling at \$0.15/M input tokens. Two years ago, the same pipeline would cost 40x more and run 10x slower. The cost/accuracy crossover point for production-grade qualitative analysis was crossed in Q1 2025.

2. Evaluation Rubric (The “Golden Dataset”)

This is the contract between the AI system and the product team. Every row defines an immutable test case that must pass before any model change, prompt edit, or pipeline modification ships to production.

2.1 Primary Evaluation Matrix

#	Input Data	Expected AI Insight	Success Criteria
1	"What I really wish is that there was a way to get a daily summary email. Like, just send me a digest every morning at 8 AM with what's due today and what's overdue." — Speaker 1	Feature Request: user_need = "User needs daily task summary," proposed_solution = "Automated morning email digest at 8 AM," confidence = 9–10 (Explicit)	supporting_quote is verbatim. confidence_score ≥ 9. source_id = "Speaker 1." Auditor entailment = True.
2	"the dashboard takes forever to load. Sometimes I'm waiting 15 to 20 seconds just to see my tasks. It's really frustrating" — Speaker 1	Pain Point: issue = "Dashboard loading unacceptably slow," impact = "15-20s wait causes context-switching," confidence = 8–9	confidence_score ≥ 7. business_impact references measurable delay. Quote is exact. Auditor = True.
3	"The color scheme is nice? I'm kidding. Sort of." — Speaker 3, multi-speaker interview with sarcasm	No feature request or pain point extracted. If extracted, confidence ≤ 3 and Auditor should reject.	System does NOT extract "color scheme customization" as a feature request. Auditor entailment = False. Zero false positives from sarcastic input.
4	3,000-word transcript, 4 speakers, French fragments, empty turns, profanity	Extracts ≥ 6/9 known pain points, ≥ 5/7 feature requests. Zero insights attributed to Speaker 0.	source_id never = "Speaker 0." Non-English text preserved. Chunking: 2+ segments, zero word loss. Coverage ≥ 67% PP, ≥ 71% FR.
5	Fabricated: "Need: AI code gen. Solution: Build a copilot." tested against: "the dashboard takes forever to load"	Auditor returns False. The insight is hallucinated — it does not follow from the segment.	verify_entailment() = False. 0 false positives across all hallucinated pairs. Precision = 100%. Non-negotiable SLA.

2.2 Why This Matrix Matters

Rows 1–2 test recall can the system find real insights? Row 3 tests sarcasm resistance, can the system avoid being tricked? Row 4 tests scale and chaos can the system handle production-grade messiness? Row 5 tests precision, the single most important guarantee. A PM will forgive a missed insight (they can search for it). They will never forgive a hallucinated insight that ships to a Jira backlog.

3. Data Pipeline Architecture

The following diagram shows the complete LangGraph agentic flow. Conditional routing at START determines single-pass vs. parallel chunked extraction based on transcript word count. Color coding: blue = input, orange = global context injection, red = adversarial auditor gate, green = human review gate, dark red = rejection path.

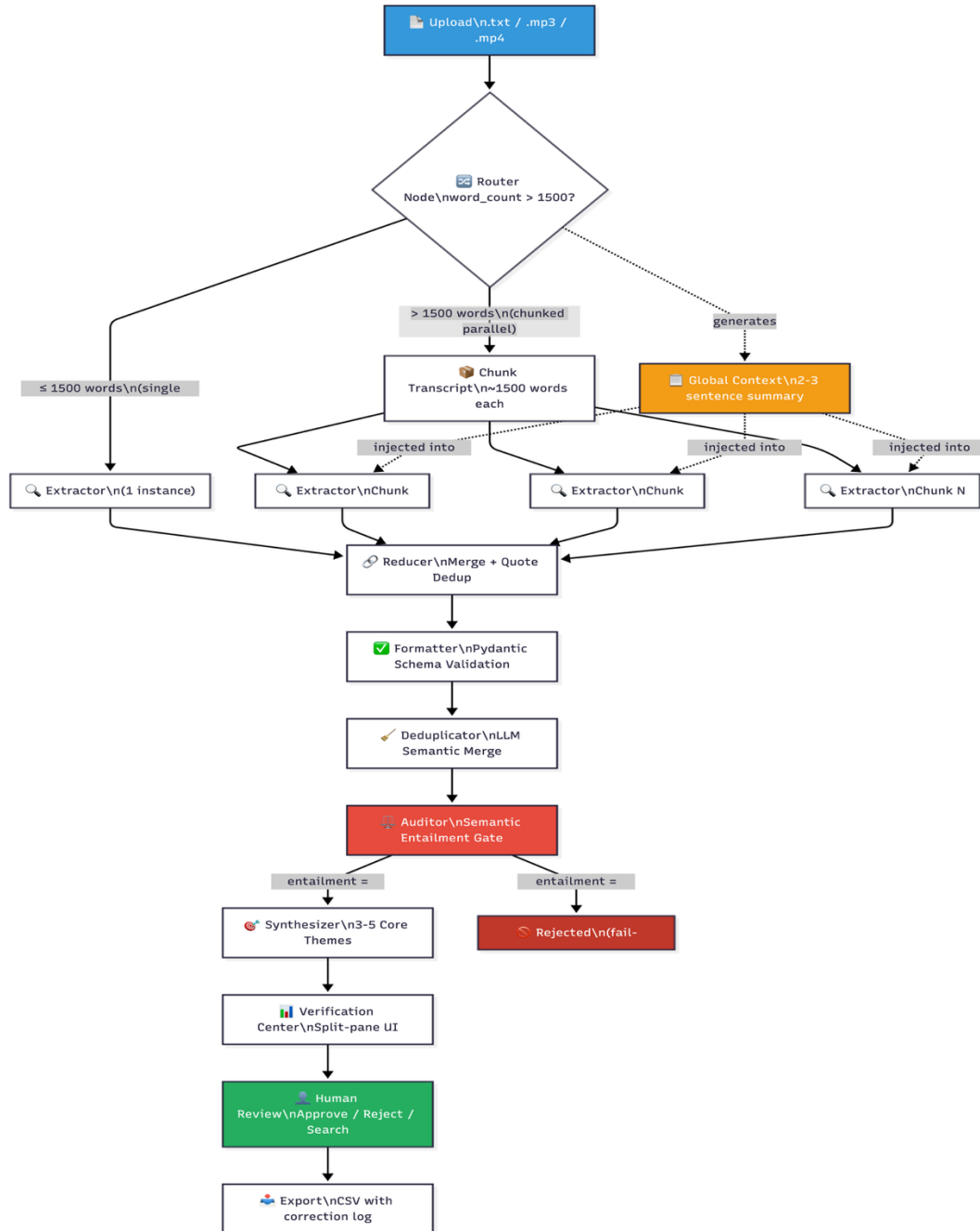


Figure 1: InsightStream End-to-End Data Pipeline

3.1 Pipeline Execution Model

The pipeline is a 6-node LangGraph StateGraph with one conditional edge (Router) that enables dynamic fan-out. It is not a chain — it is a directed acyclic graph with parallel execution and typed state passing.

Phase	Nodes	Classification	Latency
Map	Router → Extractor(s)	Probabilistic (LLM)	$O(1)$ — parallel chunks
Reduce	Reducer → Formatter	Deterministic (code)	$O(n)$ — linear in insights
Validate	Deduplicator → Auditor	Probabilistic (LLM)	$O(n)$ — one call/insight
Synthesize	Synthesizer	Probabilistic (LLM)	$O(1)$ — single call
Human Gate	Verification Center	Deterministic (UI)	User-dependent

4. Model Selection & Logic

4.1 Primary Model: Gemini 2.5 Flash

Criterion	Gemini 2.5 Flash	GPT-4o	Claude 3.5 Sonnet
Structured output	Native	Native	Native
Cost (input/1M tokens)	\$0.15	\$2.50	\$3.00
Latency (TTFT)	~200ms	~400ms	~350ms
Context window	1M tokens	128K tokens	200K tokens
Multilingual (FR/ES)	Strong	Strong	Strong
Decision	SELECTED	Too expensive at scale	Too expensive at scale

Why Flash over Pro: The Extractor task is pattern-matching with structured output, not complex reasoning. Flash achieves equivalent extraction quality at 10x lower cost. The Auditor task (binary Yes/No entailment) is similarly straightforward. Pro is available as a dropdown option for users who want it.

4.2 Approach: RAG, Not Fine-Tuning

Approach	Fit	Rationale
RAG (selected)	✓	Every query is grounded in a user-provided transcript. No external knowledge needed. The "Missing Something?" search bar performs direct RAG over the raw transcript.
Fine-Tuning	✗	Requires a labeled dataset that doesn't exist yet. The HITL correction log is building this dataset organically. Fine-tuning becomes viable at ~10,000 correction pairs (6–12 months).
Prompt Engineering + Structured Output	✓	Pydantic schemas coerce output structure. Confidence rubrics embedded in prompts constrain scoring. This is the right approach for v1.

4.3 RAG Implementation

InsightStream's RAG is intentionally simple and that's a deliberate design choice:

- No vector database. The transcript is the entire corpus. Semantic search over a single document doesn't justify embeddings + HNSW indexing overhead.
- No chunked retrieval for queries. The `query_transcript()` function passes the full transcript (truncated at 80K chars) directly to the LLM with a grounding instruction: "Based only on the following interview transcript, answer this question."
- Why this works: Gemini's 1M-token context window means even a 3-hour interview fits in a single call. The bottleneck is cost, not context length.

5. AI UX & Trust

5.1 Explainability: The “Why” Behind Every Insight

Every extracted insight shows the user three evidence layers:

1. Supporting Quote — The exact verbatim text from the transcript. Not paraphrased, not summarized. The user can Ctrl+F the quote in their original file and find it. This is the primary trust mechanism.
2. Confidence Score (1-10) — Displayed as a metric card. The rubric is transparent: 9–10 means the user explicitly asked for it; 4–6 means inference was required. Users learn to treat low-confidence insights as “worth reviewing” rather than “definitely correct.”
3. Source Attribution (source_id) — Every insight tags which speaker said it. In multi-participant interviews, this prevents the “who said what?” ambiguity that makes PMs distrust automated analysis.

The Verification Center renders these as a split-pane dashboard: structured insights on one side, with the ability to query the raw transcript (“Missing Something?”) on the other. The user is never more than one click away from the source material.

5.2 Perceived Latency Strategy

Pipeline Phase	User Sees	Duration	Technique
Upload + Preprocessing	File name appears, tab switches	< 1s	Instant UI feedback
Transcript Preview	Expandable preview with first 800 chars	< 1s	Truncation with "..."
LangGraph Processing	Spinner: "Processing {filename}..."	15–45s	st.spinner() with file name
Results Render	Toast + full UI	< 1s	st.toast() + session state

Why not streaming? InsightStream’s output is structured JSON (tables, cards, metrics), not prose. Streaming a half-formed JSON object into a card layout creates visual chaos. The correct UX is a spinner with a meaningful label, followed by an instant full render. The 15–45 second wait is acceptable because it replaces 4 hours of manual work.

5.3 The “I Don’t Know” UX

When the RAG query finds nothing relevant:

“The transcript does not contain relevant information about [query topic].”

When the Extractor finds no insights:

UI renders “No pain points identified” / “No feature requests identified” with a muted caption. The “Missing Something?” search bar is prominently positioned directly below.

The system never fabricates an answer to appear helpful. Silence is a feature.

6. Failure Modes & Guardrails

6.1 Failure Taxonomy

#	Failure Mode	Probability	Severity	Detection	Mitigation
F1	Hallucination: Extractor fabricates insight not in transcript	Medium	Critical	Auditor Node (semantic entailment)	Binary Yes/No check against transcript segment. Fail-closed: returns False on any exception. 100% precision SLA.
F2	Diarization Drift: Whisper inverts speaker labels	High	High	Speaker Assignment Step-Gate (UI)	Human reviews labels before processing. "Swap Speakers" = deterministic string replacement.
F3	Loud User Bias: One vocal participant dominates themes	Medium	Medium	source_id tracking per insight	Synthesizer prompt: "Weight by UNIQUE PARTICIPANTS, not mention count."
F4	Context Loss: LLM misses insights in long transcripts	High (60+ min)	High	Coverage metrics from eval harness	Map-Reduce chunking at 1500 words. Global context injection for coherence.
F5	Sarcasm Misinterpretation: Sarcasm extracted as genuine request	Low-Med	Medium	Auditor entailment + confidence rubric	Sarcasm fails entailment. Rubric assigns 1–3 (Vague) to ambiguous feedback. RAG search as fallback.

6.2 Rejection Criteria: When the AI Says “No”

The system rejects an insight (does not surface it to the user) when:

1. Entailment fails. `verify_entailment()` returns False. This is the primary rejection gate. Non-negotiable.
2. Schema validation fails. Pydantic rejects the LLM output (e.g., `confidence_score = 15`, missing `supporting_quote`).
3. Empty transcript. If `transcript.strip()` is empty, all nodes short-circuit. No LLM calls are made.
4. API failure after retries. 3 retries with 30/60/90s backoff exhausted. Insight rejected, not queued.
5. Duplicate quote. Identical `supporting_quote` (after normalization) → Reducer keeps first, discards duplicate.

Design principle: every rejection path defaults to “do not show.” The user can always recover missed insights via the search bar. They cannot un-see a hallucination.

7. Success Metrics

7.1 North Star Business Metric

Research Throughput Multiplier: Hours of interview content analyzed per PM per week.

Baseline	Target	Measurement
2–3 hours/week (manual synthesis of 1–2 interviews)	15–20 hours/week (6–8 interviews)	Self-reported PM survey + transcript upload volume

This metric captures the product’s core value: not “faster analysis” but “more interviews analyzed.” The outcome is better product decisions from more complete data.

7.2 Technical Metric 1: Precision @ K (Grounding Attribution Rate)

Definition	Target	Measurement	Alert
% of surfaced insights provably grounded in source transcript	100% (98%+ operational)	Auditor pass rate per run. false_positives / total must = 0	Any hallucination passing Auditor = P0

Precision @ K is chosen over raw precision because InsightStream generates a complete extraction, not a ranked list. K = all insights shown. Every single surfaced insight must be grounded.

7.3 Technical Metric 2: Latency p95

Definition	Target	Measurement	Alert
95th percentile wall-clock time from "Analyze" to results render	< 45 seconds (60-min transcript)	time.time() delta around run_agent(), logged per invocation	p95 > 90s for 3 consecutive days

The 45-second target derives from the Map-Reduce architecture: 6 chunks × ~5s parallel extraction + ~3s reduce/format + ~20s auditor (10 insights × 2s each) + ~5s synthesizer = ~33s typical, with p95 headroom for API variance.

7.4 Guardrail Metric: Manual Override Rate

Target	Alert Threshold	Signal
< 15%	> 25% sustained over 1 week	High override = Extractor produces insights passing Auditor but mismatching user judgment. Leading indicator of prompt drift.

8. Aakash Framework Compliance Checklist

Framework Element	InsightStream Implementation	Section
"Why AI, not rules?"	Semantic understanding for sarcasm, paraphrasing, multilingual, clustering	1.2
Golden Dataset (Eval Rubric)	5-row matrix: recall, sarcasm, scale, precision	2
Model Selection with rationale	Gemini 2.5 Flash: cost/speed/structured-output sweet spot	4.1
RAG vs Fine-Tuning decision	RAG now; fine-tuning when correction log reaches 10K pairs	4.2
Explainability	Verbatim quotes + confidence scores + source attribution	5.1
Perceived Latency	Spinner-based (not streaming) due to structured output	5.2
Failure Modes (5+)	Hallucination, diarization, bias, context loss, sarcasm	6.1
Rejection Criteria	5 explicit rejection paths, all fail-closed	6.2
North Star metric	Research Throughput Multiplier (hours analyzed/PM/week)	7.1
Technical metrics	Precision @ K = 100%, Latency p95 < 45s	7.2–7.3
Data pipeline visualization	Flowchart of full LangGraph DAG (Figure 1)	3